

Complete Project Documentation: Multimodal RAG

Welcome to the detailed documentation for the **Multimodal RAG** project!

1. What is this Project?

Multimodal RAG is an advanced, smart AI Chatbot.

- **Multimodal** means it can understand different types of media: Text, Audio, and Images.
- **RAG** stands for **Retrieval-Augmented Generation**. It means if you give the AI a document (like a PDF), it will read the document, "retrieve" the exact information needed from it, and then "generate" a smart answer for you.

With this project, you can:

- Type messages to chat with the AI.
- Send voice notes (Audio) and hear the AI speak back to you.
- Upload images and ask the AI what is inside the picture.
- Upload PDFs and ask questions about the document.

2. What Technologies & Tools Were Used?

We used a combination of powerful, open-source AI tools to build this:

1. Core Frameworks

- **Python**: The main programming language.
- **Streamlit**: Used to build the beautiful website (UI) where you chat with the AI.
- **LangChain**: A special tool used to connect the AI models together and manage the "chat memory".

2. AI Language Models (The Brain)

- **Ollama:** We use Ollama to run powerful AI chat models (like Llama 3) locally on the computer.

3. Computer Vision (For Images)

- **BLIP (Salesforce/blip-vqa-base):** A special AI model that looks at an uploaded image and answers questions about it.

4. Audio Processing (Voice)

- **Wav2Vec2 (HuggingFace):** Used for **Speech-to-Text**. It listens to the audio you upload and converts it into written text.
- **gTTS (Google Text-to-Speech):** Used for **Text-to-Speech**. It takes the AI's written answer and turns it into an audio file.

5. The RAG System (For PDFs)

- **Pinecone:** A "Vector Database". When we upload a PDF, we store its paragraphs here so the AI can search through it extremely fast.

3. How Does the Pipeline Work? (Step-by-Step)

Scenario A: Normal Text Chat

1. You type a message (e.g., "Hello!").
2. The message goes directly to **Ollama** (the AI brain).
3. The AI generates an answer.
4. **gTTS** converts the answer into audio.
5. The website shows you the text and plays the audio for you!

Scenario B: When You Upload a PDF (RAG Pipeline)

1. **Reading:** You upload a PDF. `pdf_handler.py` reads the PDF and chops it into small, readable paragraphs.
2. **Storing:** These paragraphs are turned into numbers (Embeddings) and saved in the **Pinecone Vector Database** (`vectorstore.py`).
3. **Asking:** You ask a question (e.g., "What does page 2 say?").

4. **Searching:** The system searches Pinecone to find the most relevant paragraphs from your PDF.
5. **Answering:** It gives those paragraphs to **Ollama**, and Ollama reads them to give you a perfect answer!

🕒 Scenario C: When You Upload an Image

1. You upload a picture and type a question (e.g., "What color is the car?").
2. The image and your question are sent to the **BLIP Model** (`vqa.py`).
3. BLIP looks at the image, reads your question, and outputs the answer.

🕒 Scenario D: When You Upload Audio

1. You upload an audio voice note.
2. **Wav2Vec2** (`audio_processor.py`) listens to it and turns your voice into text.
3. That text is sent to the AI just like a normal text message.
4. The AI replies, and the app reads it out loud for you!

📁 4. Project Folder Structure Explained

- `app.py`: The main file. It controls the website UI and buttons.
- `src/audio_processor.py`: Handles turning Voice to Text, and Text to Voice.
- `src/vqa.py`: Handles the Image AI (BLIP) to answer questions about pictures.
- `src/pdf_handler.py`: Reads PDFs and breaks them into small pieces.
- `src/vectorstore.py`: Connects to Pinecone Database to save and search the PDF pieces.
- `src/ollama_chain.py`: Connects to the Ollama AI to handle the actual chatting and memory.
- `.env`: A hidden file where you keep your secret passwords (like the Pinecone API Key).

⚙️ 5. Easy Setup Guide

If you want to run this project yourself on your computer, follow these simple steps:

```
# 1. Clone the project
```

```
git clone https://github.com/Ahmed-AI-01/Multimodal-RAG.git
cd Multimodal-RAG

# 2. Create a Virtual Environment
python -m venv venv

# 3. Activate It
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# 4. Install Everything
pip install -r requirements.txt

# 5. Set your API Key
# Create a .env file and add: PINECONE_API_KEY=your_key_here

# 6. Start the App
streamlit run app.py
```

Enjoy using your smart AI assistant! 🦾